

LibTIFF – TIFF Library and Utilities

The LibTIFF software provides support for the *Tag Image File Format* (TIFF), a widely used format for storing image data. The latest version of the TIFF specification is [TIFF File Format Specification](#).

Included in this software distribution are:

- a library, `libtiff`, for reading and writing TIFF images
- a small collection of tools for performing simple manipulations of TIFF images
- documentation for the library and tools.

The `libtiff` library, along with associated tool programs, should handle most of your needs for reading and writing TIFF images.

LibTIFF is portable software, supported on various operating systems including UNIX (Linux, BSD, MacOS X) and Windows.

Note

At libtiff v4.6.0, the source code for most TIFF tools (except `tiffinfo`, `tifftodump`, `tiffcp` and `tiffset`) has been moved to `archive/` directory and was not built. `tiff2ps` and `tiff2pdf` source code has been moved in a unsupported category, no longer built by default, but were still part of the the source distribution.

With libtiff v4.7.0 those tools were restored.

libtiff

Synopsis

```
#include <tiffio.h>
cc file.c -ltiff
```

Description

libtiff is a library for reading and writing data files encoded with the "*Tag Image File*" format, Revision 6.0 (or revision 5.0 or revision 4.0). This file format is suitable for archiving multi-color and monochromatic image data.

The library supports several compression algorithms, as indicated by the `Compression` field, including: no compression (1), CCITT 1D Huffman compression (2), CCITT Group 3 Facsimile compression (3), CCITT Group 4 Facsimile compression (4), Lempel-Ziv & Welch compression (5), baseline JPEG compression (7), word-aligned 1D Huffman compression (32771), PackBits

compression (32773). In addition, several nonstandard compression algorithms are supported: the 4-bit compression algorithm used by the *ThunderScan* program (32809) (decompression only), NeXT's 2-bit compression algorithm (32766) (decompression only), an experimental LZ-style algorithm known as Deflate (32946), and an experimental CIE LogLuv compression scheme designed for images with high dynamic range (32845 for LogL and 32845 for LogLuv). Directory information may be in either little- or big-endian byte order; byte swapping is automatically done by the library. Data bit ordering may be either Most Significant Bit (MSB) to Least Significant Bit (LSB) or LSB to MSB. Finally, the library does not support files in which the `BitsPerSample`, `Compression`, `MinSampleValue`, or `MaxSampleValue` fields are defined differently on a per-sample basis (in Rev. 6.0 the `Compression` tag is not defined on a per-sample basis, so this is immaterial).

Data types

The library makes extensive use of C typedefs to promote portability. Two sets of typedefs are used, one for communication with clients of the library and one for internal data structures and parsing of the TIFF format. The following typedefs are exposed to users either through function definitions or through parameters passed through the varargs interfaces.

```
typedef int64_t tmsize_t;    /* signed size type (int32_t on 32-bit
                             platforms) */
typedef void* thandle_t;    /* client data handle */
typedef uint64_t toff_t;    /* file offset */
typedef uint32_t tdir_t;    /* directory index */
```

The following typedefs are deprecated and should be replaced by their defining counterparts.

```
typedef uint32_t ttag_t;    /* directory tag */
typedef uint16_t tsample_t; /* sample number */
typedef uint32_t tstrile_t; /* strip or tile number */
typedef tstrile_t tstrip_t; /* strip number */
typedef tstrile_t ttile_t;  /* tile number */
typedef tmsize_t tsize_t;   /* i/o size in bytes */
typedef void *tdata_t;     /* image data ref */
```

Note that `tstrip_t`, `ttile_t`, and `tsize_t` are constrained to be no more than 32-bit quantities by 32-bit fields they are stored in in the TIFF image. Likewise `tsample_t` is limited by the 16-bit field used to store the `SamplesPerPixel` tag.

`tdir_t` constrains the maximum number of IFDs that may appear in an image and may be an arbitrary size (w/o penalty). Starting with libtiff 4.5.0, `tdir_t` is a 32-bit unsigned integer. Previously, it was a 16-bit unsigned integer.

`ttag_t` must be either `int`, unsigned `int`, pointer, or double because the library uses a varargs interface and C restricts the type of the parameter before an ellipsis to be a promoted type. `toff_t` is defined as `uint64_t` because TIFF file offsets are (unsigned) 32-bit quantities, and BigTIFF file offsets are unsigned 64-bit quantities. A signed value is used because some

interfaces return -1 on error. Finally, note that user-specified data references are passed as opaque handles and only cast at the lowest layers where their type is presumed.

List of routines

The following routines are part of the library. Consult specific manual pages for details on their operation; on most systems doing **man function-name** will work.

Libtiff functions

Name	Description
<u>TIFFAccessTagMethods()</u>	provides read/write access to the TIFFTagMethods within the TIFF structure to application code without giving access to the private TIFF structure
<u>TIFFCheckpointDirectory()</u>	writes the current state of the directory
<u>TIFFCheckTile()</u>	very x,y,z,sample is within image
<u>TIFFCIELabToRGBInit()</u>	initialize CIE L*a*b* 1976 to RGB conversion state
<u>TIFFCIELabToXYZ()</u>	perform CIE L*a*b* 1976 to CIE XYZ conversion
<u>TIFFCleanup()</u>	auxiliary function to free the TIFF structure
<u>TIFFClientdata()</u>	return open file's clientdata handle, which represents the file descriptor used within libtiff.
<u>TIFFClientOpen()</u>	open a file for reading or writing
<u>TIFFClientOpenExt()</u>	open a file for reading or writing with options, such as re-entrant error and warning handlers may be passed
<u>TIFFClose()</u>	close a previously opened TIFF file
<u>TIFFComputeStrip()</u>	return strip containing y,sample
<u>TIFFComputeTile()</u>	return tile containing x,y,z,sample
<u>TIFFCreateCustomDirectory()</u>	setup for a <i>custom</i> directory in a open TIFF file
<u>TIFFCreateDirectory()</u>	setup for a directory in a open TIFF file
<u>TIFFCreateEXIFDirectory()</u>	setup for a <i>EXIF</i> custom directory in a open TIFF file within a TIFF tag
<u>TIFFCreateGPSDirectory()</u>	setup for a <i>GPS</i> custom directory in a open TIFF file within a TIFF tag
<u>TIFFCurrentDirectory()</u>	return index of current directory
<u>TIFFCurrentDirOffset()</u>	return file offset of the current directory (instead of an index)
<u>TIFFCurrentRow()</u>	return index of current scanline
<u>TIFFCurrentStrip()</u>	return index of current strip
<u>TIFFCurrentTile()</u>	return index of current tile
<u>TIFFDataWidth()</u>	return the size of TIFF data types
<u>TIFFDefaultStripSize()</u>	return number of rows for a reasonable-sized strip according to the current settings of the ImageWidth,

Libtiff functions

Name	Description
	BitsPerSample and SamplesPerPixel, tags and any compression-specific requirements
<u>TIFFDefaultTileSize()</u>	return pixel width and height of a reasonable-sized tile; suitable for setting up the TileWidth and TileLength tags
<u>TIFFDeferStrileArrayWriting()</u>	is an advanced writing function to control when/where the [Strip/Tile][Offsets/ByteCounts] arrays are written into the file, and must be used in a particular sequence together with TIFFForceStrileArrayWriting() (see description)
<u>TIFFError()</u>	library-wide error handling function printing to stderr
<u>TIFFErrorExt()</u>	user-specific library-wide error handling function that can be passed a file handle, which is set to the open TIFF file within libtiff
<u>TIFFErrorExtR()</u>	user-specific re-entrant library error handling function, to which its TIFF structure is passed containing the pointer to a user-specific data object
<u>TIFFFdOpen()</u>	open a file for reading or writing
<u>TIFFFdOpenExt()</u>	open a file for reading or writing with options, such as re-entrant error and warning handlers may be passed
<u>TIFFfieldDataType()</u>	get data type from field information
<u>TIFFfieldIsAnonymous()</u>	returns if field was unknown to libtiff and has been auto-registered
<u>TIFFfieldName()</u>	get field name from field information
<u>TIFFfieldPassCount()</u>	get whether to pass a value count to Get/SetField
<u>TIFFfieldReadCount()</u>	get number of values to be read from field
<u>TIFFfieldSetGetCountSize()</u>	returns size of count parameter of <u>TIFFSetField()</u> and <u>TIFFGetField()</u>
<u>TIFFfieldSetGetSize()</u>	return data size in bytes of the field data type used for libtiff internal storage.
<u>TIFFfieldTag()</u>	get tag value from field information
<u>TIFFfieldWithName()</u>	get field information given field name
<u>TIFFfieldWithTag()</u>	get field information given tag
<u>TIFFfieldWriteCount()</u>	get number of values to be written to field
<u>TIFFfileName()</u>	return name of open file
<u>TIFFfileno()</u>	return open file descriptor
<u>TIFFfindCODEC()</u>	find standard codec for the specific scheme
<u>TIFFfindField()</u>	get field information given tag and data type

Libtiff functions

Name	Description
<u>TIFFFlush()</u>	flush all pending writes
<u>TIFFFlushData()</u>	flush pending data writes
<u>TIFFForceStrileArrayWriting()</u>	is an advanced writing function that writes the [Strip/Tile][Offsets/ByteCounts] arrays at the end of the file (see description)
<u>TIFFFreeDirectory()</u>	release storage associated with a directory
<u>TIFFGetBitRevTable()</u>	return bit reversal table
<u>TIFFGetClientInfo()</u>	returns a pointer to the data of the named entry in the clientinfo-list
<u>TIFFGetCloseProc()</u>	returns a pointer to file close method
<u>TIFFGetConfiguredCODECs()</u>	gets list of configured codecs, both built-in and registered by user
<u>TIFFGetField()</u>	return tag value in current directory
<u>TIFFGetFieldDefaulted()</u>	return tag value in current directory with default value set if the value is not already set and a default is defined
<u>TIFFGetMapFileProc()</u>	returns a pointer to memory mapping method
<u>TIFFGetMode()</u>	return open file mode
<u>TIFFGetReadProc()</u>	returns a pointer to file read method
<u>TIFFGetSeekProc()</u>	returns a pointer to file seek method
<u>TIFFGetSizeProc()</u>	returns a pointer to file size requesting method
<u>TIFFGetStrileByteCount()</u>	return value of the TileByteCounts/StripByteCounts array for the specified tile/strile
<u>TIFFGetStrileByteCountWithErr()</u>	same as <i>TIFFGetStrileByteCount()</i> and additionally provides an error return
<u>TIFFGetStrileOffset()</u>	return value of the TileOffsets/StripOffsets array for the specified tile/strile
<u>TIFFGetStrileOffsetWithErr()</u>	same as <i>TIFFGetStrileOffset()</i> and additionally provides an error return
<u>TIFFGetTagListCount()</u>	return number of entries in the custom tag list
<u>TIFFGetTagListEntry()</u>	return tag number of the (n.th - 1) entry within the custom tag list
<u>TIFFGetUnmapFileProc()</u>	returns a pointer to memory unmapping method
<u>TIFFGetVersion()</u>	return library version string
<u>TIFFGetWriteProc()</u>	returns a pointer to file write method
<u>TIFFIsBigEndian()</u>	returns a non-zero value if the file is BigEndian and zero if the file is LittleEndian
<u>TIFFIsBigTIFF()</u>	returns a non-zero value if the file is in BigTIFF style
<u>TIFFIsByteSwapped()</u>	return true if image data is byte-swapped

Libtiff functions

Name	Description
<u>TIFFIsCODECConfigured()</u>	check, whether we have working codec
<u>TIFFIsMSB2LSB()</u>	return true if image data is being returned with bit 0 as the most significant bit
<u>TIFFIsTiled()</u>	return true if image data is tiled
<u>TIFFIsUpSampled()</u>	returns a non-zero value if image data returned through the read interface Routines is being up-sampled
<u>TIFFLastDirectory()</u>	returns a non-zero value if the current directory is the last directory in the file; otherwise zero is returned
<u>TIFFMergeFieldInfo()</u>	adds application defined TIFF tags to the list of known libtiff tags
<u>TIFFNumberOfDirectories()</u>	return number of directories in a file
<u>TIFFNumberOfStrips()</u>	return number of strips in an image
<u>TIFFNumberOfTiles()</u>	return number of tiles in an image
<u>TIFFOpen()</u>	open a file for reading or writing
<u>TIFFOpenExt()</u>	open a file for reading or writing with options, such as re-entrant error and warning handlers may be passed
<u>TIFFOpenW()</u>	opens a TIFF file with a Unicode filename, for read/writing
<u>TIFFOpenWExt()</u>	opens a TIFF file with a Unicode filename, for read/writing with options, such as re-entrant error and warning handlers may be passed
<u>TIFFOpenOptionsAlloc()</u>	allocates memory for <u>TIFFOpenOptions</u> opaque structure
<u>TIFFOpenOptionsFree()</u>	releases the allocated memory for <u>TIFFOpenOptions</u>
<u>TIFFOpenOptionsSetMaxSingleMemAlloc()</u>	limits the maximum single memory allocation within libtiff
<u>TIFFOpenOptionsSetMaxCumulatedMemAlloc()</u>	limits the maximum cumulated memory allocation within libtiff
<u>TIFFOpenOptionsSetErrorHandlerExtR()</u>	setup of a user-specific and per-TIFF handle (re-entrant) error handler
<u>TIFFOpenOptionsSetWarningHandlerExtR()</u>	setup of a user-specific and per-TIFF handle (re-entrant) warning handler
<u>TIFFPrintDirectory()</u>	print description of the current directory
<u>TIFFRasterScanlineSize()</u>	returns the size in bytes of a complete decoded and packed raster scanline
<u>TIFFRasterScanlineSize64()</u>	return size as <code>uint64_t</code>

Libtiff functions

Name	Description
<u>TIFFRawStripSize()</u>	return number of bytes in a raw strip
<u>TIFFRawStripSize64()</u>	return number of bytes in a raw strip as <code>uint64_t</code>
<u>TIFFReadBufferSetup()</u>	specify i/o buffer for reading
<u>TIFFReadCustomDirectory()</u>	read the custom directory from the given offset and set the context of the TIFF-handle tif to that custom directory
<u>TIFFReadDirectory()</u>	read the next directory
<u>TIFFReadEncodedStrip()</u>	read and decode a strip of data
<u>TIFFReadEncodedTile()</u>	read and decode a tile of data
<u>TIFFReadEXIFDirectory()</u>	read the EXIF directory from the given offset and set the context of the TIFF-handle tif to that EXIF directory
<u>TIFFReadFromUserBuffer()</u>	replaces the use of <u>TIFFReadEncodedStrip()</u> / <u>TIFFReadEncodedTile()</u> when the user can provide the buffer for the input data
<u>TIFFReadGPSDirectory()</u>	read the GPS directory from the given offset and set the context of the TIFF-handle tif to that GPS directory
<u>TIFFReadRawStrip()</u>	read a raw strip of data
<u>TIFFReadRawTile()</u>	read a raw tile of data
<u>TIFFReadRGBAImage()</u>	read an image into a fixed format raster
<u>TIFFReadRGBAImageOriented()</u>	works like <u>TIFFReadRGBAImage()</u> except that the user can specify the raster origin position
<u>TIFFReadRGBAStrip()</u>	reads a single strip of a strip-based image into memory, storing the result in the user supplied RGBA raster
<u>TIFFReadRGBAStripExt()</u>	same as <u>TIFFReadRGBAStrip()</u> but providing the parameter <i>stop_on_error</i>
<u>TIFFReadRGBATile()</u>	reads a single tile of a tile-based image into memory, storing the result in the user supplied RGBA raster
<u>TIFFReadRGBATileExt()</u>	same as <u>TIFFReadRGBATile()</u> but providing the parameter <i>stop_on_error</i>
<u>TIFFReadScanline()</u>	read and decode a row of data
<u>TIFFReadTile()</u>	read and decode a tile of data
<u>TIFFRegisterCODEC()</u>	override standard codec for the specific scheme
<u>TIFFReverseBits()</u>	reverse bits in an array of bytes
<u>TIFFRewriteDirectory()</u>	operates similarly to <u>TIFFWriteDirectory()</u> , but can be called with directories previously read or

Libtiff functions

Name	Description
	written that already have an established location in the file and places it at the end of the file
<u>TIFFRGBAImageBegin()</u>	setup decoder state for TIFFRGBAImageGet
<u>TIFFRGBAImageEnd()</u>	release TIFFRGBAImage decoder state
<u>TIFFRGBAImageGet()</u>	read and decode an image
<u>TIFFRGBAImageOK()</u>	is image readable by TIFFRGBAImageGet
<u>TIFFScanlineSize()</u>	return size of a scanline
<u>TIFFScanlineSize64()</u>	return size of a scanline as <code>uint64_t</code>
<u>TIFFSetClientdata()</u>	set open file's clientdata (file descriptor/handle), and return previous value
<u>TIFFSetClientInfo()</u>	adds or replaces an entry in the clientinfo-list
<u>TIFFSetCompressionScheme()</u>	set compression scheme
<u>TIFFSetDirectory()</u>	set the current directory
<u>TIFFSetErrorHandler()</u>	set error handler function
<u>TIFFSetErrorHandlerExt()</u>	set error handler function with a file handle as parameter
<u>TIFFSetField()</u>	set a tag's value in the current directory
<u>TIFFSetFileName()</u>	sets the file name in the TIFF-structure and returns the old file name
<u>TIFFSetFileno()</u>	overwrites a copy of the open file's I/O descriptor, and return previous value (refer to detailed description)
<u>TIFFSetMode()</u>	sets the <code>libtiff</code> open mode in the TIFF-structure and returns the old mode
<u>TIFFSetSubDirectory()</u>	set the current directory
<u>TIFFSetTagExtender()</u>	is used to register the merge function for user defined tags as an extender callback with <code>libtiff</code>
<u>TIFFSetupStrips()</u>	setup or reset strip parameters and strip array memory
<u>TIFFSetWarningHandler()</u>	set warning handler function
<u>TIFFSetWarningHandlerExt()</u>	set warning handler function with a file handle as parameter
<u>TIFFSetWriteOffset()</u>	set current write offset
<u>TIFFStripSize()</u>	return size of a strip
<u>TIFFStripSize64()</u>	return equivalent size for a strip of data as <code>uint64_t</code>
<u>TIFFSwabArrayOfDouble()</u>	swap bytes of an array of doubles
<u>TIFFSwabArrayOfFloat()</u>	swap bytes of an array of floats
<u>TIFFSwabArrayOfLong()</u>	swap bytes of an array of longs
<u>TIFFSwabArrayOfLong8()</u>	swap bytes of an array of <code>uint64_t</code>

Libtiff functions

Name	Description
<u>TIFFSwabArrayOfShort()</u>	swap bytes of an array of shorts
<u>TIFFSwabArrayOfTriples()</u>	swap the first and third byte of each triple within an array of bytes
<u>TIFFSwabDouble()</u>	swap bytes of double
<u>TIFFSwabFloat()</u>	swap bytes of float
<u>TIFFSwabLong()</u>	swap bytes of long
<u>TIFFSwabLong8()</u>	swap bytes of long long (uint64_t)
<u>TIFFSwabShort()</u>	swap bytes of short
<u>TIFFTileRowSize()</u>	return size of a row in a tile
<u>TIFFTileRowSize64()</u>	return size of a row in a tile as uint64_t
<u>TIFFTileSize()</u>	return size of a tile
<u>TIFFTileSize64()</u>	return size of a tile as uint64_t
<u>TIFFUnlinkDirectory()</u>	unlink the specified directory from the directory chain
<u>TIFFUnregisterCODEC()</u>	unregisters the codec
<u>TIFFUnsetField()</u>	clear the contents of the field in the internal structure
<u>TIFFVGetField()</u>	return tag value in current directory
<u>TIFFVGetFieldDefaulted()</u>	return tag value in current directory
<u>TIFFVSetField()</u>	set a tag's value in the current directory
<u>TIFFVStripSize()</u>	return number of bytes in a strip
<u>TIFFVStripSize64()</u>	return number of bytes in a strip with <i>nrows</i> rows of data as uint64_t
<u>TIFFVTileSize()</u>	returns the number of bytes in a row-aligned tile with <i>nrows</i> of data
<u>TIFFVTileSize64()</u>	returns the number of bytes in a row-aligned tile with <i>nrows</i> of data a uint64_t number
<u>TIFFWarning()</u>	library-wide warning handling function printing to <code>stderr</code>
<u>TIFFWarningExt()</u>	user-specific library-wide warning handling function that can be passed a file handle, which is set to the open TIFF file within <code>libtiff</code>
<u>TIFFWarningExtR()</u>	user-specific re-entrant library warning handling function, to which its TIFF structure is passed containing the pointer to a user-specific data object
<u>TIFFWriteBufferSetup()</u>	sets up the data buffer used to write raw (encoded) data to a file
<u>TIFFWriteCheck()</u>	verify file is writable and that the directory information is setup properly

Libtiff functions

Name	Description
<u>TIFFWriteCustomDirectory()</u>	write the current custom directory (also EXIF or GPS) to file
<u>TIFFWriteDirectory()</u>	write the current directory
<u>TIFFWriteEncodedStrip()</u>	compress and write a strip of data
<u>TIFFWriteEncodedTile()</u>	compress and write a tile of data
<u>TIFFWriteRawStrip()</u>	write a raw strip of data
<u>TIFFWriteRawTile()</u>	write a raw tile of data
<u>TIFFWriteScanline()</u>	write a scanline of data
<u>TIFFWriteTile()</u>	compress and write a tile of data
<u>TIFFXYZToRGB()</u>	perform CIE XYZ to RGB conversion
<u>TIFFYCbCrToRGB()</u>	perform YCbCr to RGB conversion
<u>TIFFYCbCrToRGBInit()</u>	initialize YCbCr to RGB conversion state

Libtiff auxiliary functions

Name	Description
<u>TIFFCheckMalloc()</u>	checking for integer overflow before dynamically allocate memory buffer
<u>TIFFCheckRealloc()</u>	checking for integer overflow before dynamically reallocate memory buffer
<u>TIFFClampDoubleToUInt32()</u>	clamps double values into the range of <code>uint32_t</code> (i.e. 0 .. 0xFFFFFFFF)
<u>TIFFfree()</u>	free memory buffer
<u>TIFFGetExifFields()</u>	return a pointer to the <code>libtiff</code> internal definition list of the EXIF tags
<u>TIFFGetGpsFields()</u>	return a pointer to the <code>libtiff</code> internal definition list of the GPS tags
<u>TIFFmalloc()</u>	dynamically allocate memory buffer
<u>TIFFmemcmp()</u>	compare contents of the memory buffers
<u>TIFFmemcpy()</u>	copy contents of the one buffer to another
<u>TIFFmemset()</u>	fill memory buffer with a constant byte
<u>TIFFMultiply32()</u>	checks for an integer overflow of the multiplication result of <code>uint32_t</code> and return the multiplication result or 0 if an overflow would happen
<u>TIFFMultiply64()</u>	checks for an integer overflow of the multiplication result of <code>uint64_t</code> and return the multiplication result or 0 if an overflow would happen
<u>TIFFrealloc()</u>	dynamically reallocate memory buffer
<u>TIFFRewriteField()</u>	Rewrite a field in the directory on disk without regard to updating the TIFF directory structure in memory

Tag usage

For a table of TIFF tags recognized by the library refer to [LibTIFF Coverage of the TIFF 6.0 Specification](#).

"Pseudo tags"

In addition to the normal TIFF tags the library supports a collection of tags whose values lie in a range outside the valid range of TIFF tags. These tags are termed *pseudo-tags* and are used to control various codec-specific functions within the library. The table below summarizes the defined pseudo-tags.

libtiff supported tags

Tag name	Codec	R/W	Library Use/Notes
TIFFTAG_FAXMODE	G3	R/W	general codec operation
TIFFTAG_FAXFILLFUNC	G3/G4	R/W	bitmap fill function
TIFFTAG_JPEGQUALITY	JPEG	R/W	compression quality control
TIFFTAG_JPEGCOLORMODE	JPEG	R/W	control colorspace conversions
TIFFTAG_JPEGTABLESMODE	JPEG	R/W	control contents of JPEGTables tag
TIFFTAG_ZIPQUALITY	Deflate	R/W	compression quality level
TIFFTAG_PIXARLOGDATAFMT	PixarLog	R/W	user data format
TIFFTAG_PIXARLOGQUALITY	PixarLog	R/W	compression quality level
TIFFTAG_SGILOGDATAFMT	SGILog	R/W	user data format

TIFFTAG_FAXMODE:

Control the operation of the Group 3 codec. Possible values (independent bits that can be combined by or'ing them together) are:

FAXMODE_CLASSIC:

(enable old-style format in which the RTC is written at the end of the last strip),

FAXMODE_NORTC:

(opposite of FAXMODE_CLASSIC; also called FAXMODE_CLASSF),

FAXMODE_NOEOL:

(do not write EOL codes at the start of each row of data),

FAXMODE_BYTEALIGN:

(align each encoded row to an 8-bit boundary),

FAXMODE_WORDALIGN:

(align each encoded row to an 16-bit boundary),

The default value is dependent on the compression scheme; this pseudo-tag is used by the various G3 and G4 codecs to share code.

TIFFTAG_FAXFILLFUNC:

Control the function used to convert arrays of black and white runs to packed bit arrays. This hook can be used to image decoded scanlines in multi-bit depth rasters (e.g. for display in colormap mode) or for other purposes. The default value is a pointer to a builtin function that images packed bilevel data.

TIFFTAG_IPTCNEWSPHOTO:

Tag contains image metadata per the IPTC newphoto spec: Headline, captioning, credit, etc... Used by most wire services.

TIFFTAG_PHOTOSHOP:

Tag contains Photoshop captioning information and metadata. Photoshop uses in parallel and redundantly alongside **IPTCNEWSPHOTO** information.

TIFFTAG_JPEGQUALITY:

Control the compression quality level used in the baseline algorithm. Note that quality levels are in the range 0-100 with a default value of 75.

TIFFTAG_JPEGCOLORMODE:

Control whether or not conversion is done between RGB and YCbCr colorspace. Possible values are:

JPEGCOLORMODE_RAW:

(do not convert), and

JPEGCOLORMODE_RGB:

(convert to/from RGB)

The default value is **JPEGCOLORMODE_RAW**.

TIFFTAG_JPEGTABLESMODE:

Control the information written in the **JPEGTables** tag. Possible values (independent bits that can be combined by or'ing them together) are:

JPEGTABLESMODE_QUANT:

(include quantization tables), and

JPEGTABLESMODE_HUFF:

(include Huffman encoding tables).

The default value is JPEGTABLESMODE_QUANT | JPEGTABLESMODE_HUFF.

TIFFTAG_ZIPQUALITY:

Control the compression technique used by the Deflate codec. Quality levels are in the range 1-9 with larger numbers yielding better compression at the cost of more computation. The default quality level is 6 which yields a good time-space tradeoff.

TIFFTAG_PIXARLOGDATAFMT:

Control the format of user data passed *in* to the PixarLog codec when encoding and passed *out* from when decoding. Possible values are:

PIXARLOGDATAFMT_8BIT:

for 8-bit unsigned pixels,

PIXARLOGDATAFMT_8BITABGR:

for 8-bit unsigned ABGR-ordered pixels,

PIXARLOGDATAFMT_11BITLOG:

for 11-bit log-encoded raw data,

PIXARLOGDATAFMT_12BITPICIO:

for 12-bit PICIO-compatible data,

PIXARLOGDATAFMT_16BIT:

for 16-bit signed samples, and

PIXARLOGDATAFMT_FLOAT:

for 32-bit IEEE floating point samples.

TIFFTAG_PIXARLOGQUALITY:

Control the compression technique used by the PixarLog codec. This value is treated identically to **TIFFTAG_ZIPQUALITY**; see the above description.

TIFFTAG_SGILOGDATAFMT:

Control the format of client data passed *in* to the SGILog codec when encoding and passed *out* from when decoding. Possible values are:

SGILOGDATAFMT_FLTXYZ:

for converting between LogLuv and 32-bit IEEE floating valued XYZ pixels,

SGILOGDATAFMT_16BITLUV:

for 16-bit encoded Luv pixels,

SGILOGDATAFMT_32BITRAW: SGILOGDATAFMT_24BITRAW:

for no conversion of data,

SGILOGDATAFMT_8BITRGB:

for returning 8-bit RGB data (valid only when decoding LogLuv-encoded data),

SGILOGDATAFMT_FLTY:

for converting between LogL and 32-bit IEEE floating valued Y pixels,

SGILOGDATAFMT_16BITL:

for 16-bit encoded L pixels, and

SGILOGDATAFMT_8BITGRY:

for returning 8-bit greyscale data (valid only when decoding LogL-encoded data).